

ResearchLens: Implementation Plan

Goal: for any research question, return an answer where **every sentence is backed by a real passage from a real paper**, checked by an NLI model. If that is not possible, say "insufficient evidence" and explain why.

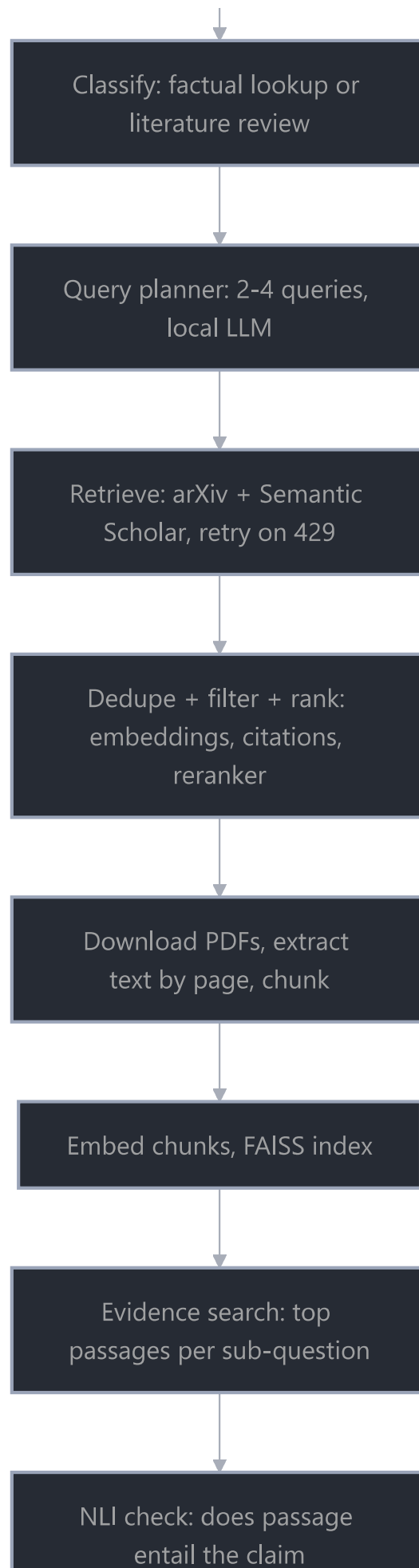
Test question (must pass at the end): *"Which paper introduced the Transformer architecture, and what was its key idea?"* Expected: **Attention Is All You Need** (Vaswani et al., 2017, arXiv 1706.03762); key idea: self-attention replaces recurrence and convolution.

1. Why it fails today

#	Problem	Where
1	Two pipelines. The UI runs the browser one and never reads the backend result	src/App.tsx L156 → api.generateReport()
2	Browser pipeline has no passages, so every citation is "unsupported"	api.ts, verificationEngine.ts
3	Hardcoded fake reports and a fake DOI	api.ts buildPlaceholderReport
4	Query generation uses Gemini only. With the local LLM it falls back to junk keywords ("paper introduced transformer architecture key idea")	paper_retrieval.py
5	Semantic Scholar errors are swallowed and return []	_fetch_s2_for_query
6	No relevance ranking. Withdrawn and off-topic papers get in	retrieve_papers
7	Report is written first, evidence checked after	research_workflow.py
8	Tiny local model (0.5B) writes a big nested JSON; the normalizer fills gaps with invented values	local_llm_service.py
9	NLI heuristic fallback returns ENTAILS by default; broken number regex	nli_verifier.py
10	Passages are never saved (passages_json empty)	research_workflow.py

2. Target pipeline

Question





Key change: **evidence first, writing last**. The LLM never invents a claim; it only rephrases verified passages.

3. What to use

Layer	Use	Why
Backend	FastAPI, Pydantic v2, SQLAlchemy, SQLite (later PostgreSQL)	Already in place, keep
Local LLM runtime	Ollama (or llama.cpp server) with qwen2.5:3b-instruct (or 1.5b if VRAM is under 4 GB)	Quantized, fits small GPUs, supports forced JSON output. No custom torch loading code

LLM job	Query planning, sub-question splitting, final rephrasing only. Never fact generation	Small models are unreliable at facts
Embeddings	BAAI/bge-small-en-v1.5 (or keep all-MiniLM-L6-v2)	Better retrieval, still small
Reranker (new)	cross-encoder/ms-marco-MiniLM-L-6-v2	Fixes ranking of abstracts and passages
NLI	MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli (upgrade from deberta-v3-small), run on GPU	More accurate entailment
Vector search	FAISS (IndexFlatIP, normalized vectors)	Already in place
PDF	PyMuPDF (fitz), keep page numbers	Already in place
Retrieval APIs	arXiv API + Semantic Scholar Graph API (get a free API key)	arXiv: keep to about 1 request per 3 seconds
Retries	tenacity (exponential backoff, honor Retry-After)	Fixes silent 429 failures
Frontend	React + TS + Vite (keep), polling GET /api/research/{id}	Single source of truth
Tests	pytest, pytest-asyncio, respx (mock HTTP)	Fast unit tests, plus one live e2e
Logging	Python logging with a per-job job_id field	Needed to debug stages

Install additions: tenacity, respx, pytest-asyncio, ollama (python client) or plain httpx to the Ollama REST API.

4. Data contracts (define first)

```

class Passage: id, paper_id, page, section, text, embedding_idx
class Evidence: claim_id, passage_id, nli_label, entail_prob, contra_prob, rerank_score
class Claim: id, text, sub_question, evidence: list[Evidence], status # verified|partial|unsuppo
class StageStat: name, started_at, duration_ms, in_count, out_count, error
class JobResult: papers, passages_total, claims, answer_text, status, stage_stats, failure_reason

```

Rules:

- Every number shown in the UI/PDF (papers, claims, evidence) is computed from JobResult, once.
- Every [n] in the answer maps to a Claim → Evidence → Passage → Paper. No orphan citations.
- Store stage_stats for every run. This is how you debug in 10 seconds.

5. Phases

Phase 0: Clean the repo (30 min)

- Delete nested `genai project/genai project/`, `dist/`, `__pycache__`, `.pytest_cache`, both `researchlens.db` files.
- Make `.gitignore` cover them; run `git init` if not already.
- Never commit `.env`. Remove `VITE_GEMINI_API_KEY` from the frontend (keys live in `backend/.env` only).
- **Done when:** one `backend/`, one `src/`, repo under ~5 MB.

Phase 1: Single source of truth in the UI (2-3 h)

Files: `src/App.tsx`, `src/services/api.ts`, `src/utils/exportPdf.ts`.

1. Set `VITE_API_URL=http://localhost:8000`. If the backend is down, show an error; do **not** fall back to browser logic.
 2. Delete `buildPlaceholderReport`, `generateReport`, `fetchArxivPapers`, `buildSearchQueriesWithGemini`, all canned reports, and `verificationEngine.ts` (and `test_verification_engine.mjs`).
 3. `handleStartResearch`: `POST /api/research` → store id → poll `GET /api/research/{id}` every 2 s until status is completed, insufficient_evidence or failed.
 4. Render badges, report, sources and PDF from that one `ResearchInvestigation` object. Sources list = `report.references` from the backend.
 5. Show real per-stage progress from `pipeline` and `stage_stats`.
- **Done when:** the numbers in the PDF match the database row for the same job.

Phase 2: Observability (1-2 h)

Files: `research_workflow.py`.

- Wrap each stage: record `StageStat` (counts in/out, duration, exception text).
- Replace silent return `[]` with explicit errors recorded in `stage_stats` (e.g. S2: HTTP 429 after 4 retries).
- Add `GET /api/research/{id}/debug` returning stage stats, queries used, titles retrieved, passages per paper.
- **Done when:** a failed run tells you which stage failed and why.

Phase 3: Retrieval (1 day)

Files: `paper_retrieval.py`, new `query_planner.py`, new `ranker.py`.

3a. Question classifier. Keep the regex in `local_llm_service.py` but move it to `query_planner.py`. Classes: `factual_lookup`, `literature_review`.

3b. Query planner (local LLM, JSON-constrained). Output:

```
{
  "queries": ["transformer self-attention", "attention is all you need"],
  "title_guesses": ["Attention Is All You Need"]
}
```

```
title_guesses = [ "Attention is All You Need" ],
"sub_questions": [ "Which paper introduced the Transformer?", "What is its key idea?" ] }
```

Prompt rules: 2-5 words per query, no question words, include named papers or authors, no filler terms.

Fallback if the LLM fails: stop-word removal **plus** removing meta words (paper, introduced, key, idea, propose, study, approach, method).

3c. Fetching.

- arXiv: search_query=all:"<phrase>" (quote phrases); for title_guesses use ti:"<title>". URL-encode via `httpx params=`. Add a 3 s delay between arXiv calls.
- Semantic Scholar: `/graph/v1/paper/search` with `x-api-key`; fields: `title,authors,year,venue,externalIds,abstract,openAccessPdf,citationCount,influentialCitationCount`.
- tenacity retry: 4 attempts, exponential backoff, respect `Retry-After`.
- Cache responses in SQLite keyed by (source, query) for 24 h (saves rate limit while testing).

3d. Filter and rank (ranker.py).

1. Dedupe by arXiv ID, DOI, then normalized title.
 2. Drop: title contains "withdrawn", empty abstract, year after today.
 3. $\text{Score} = 0.6 * \text{reranker}(\text{question}, \text{title} + \text{abstract}) + 0.25 * \log\text{-scaled citationCount} + 0.15 * \text{title-match bonus}$.
 4. For `factual_lookup`, strongly boost exact title matches and highly cited originals.
 5. Keep top N: Quick 6, Standard 12, Deep 24 (match the UI labels).
- **Done when:** for the test question, arXiv 1706.03762 is in the top 3, and no "withdrawn" or off-topic papers are in the top 10.

Phase 4: Text extraction and indexing (1 day)

Files: `pdf_extractor.py`, `vector_store.py`, `embeddings.py`.

1. Download PDFs (timeout 30 s, max 25 MB, 4 workers, cache in `PDF_CACHE_DIR`).
 2. Extract text per page with PyMuPDF; remove headers and footers repeated on most pages; cut the references section.
 3. Chunk by **3-sentence windows with 1 sentence overlap**, tagged with page and section.
 4. **If a PDF fails, fall back to the abstract as one passage** and mark `source_kind="abstract"`. Never leave a paper with 0 passages silently.
 5. Persist passages to `papers.passages_json` (this is currently never written).
 6. Embed with `bge-small` (normalized), build FAISS `IndexFlatIP`.
- **Done when:** the debug endpoint shows more than 100 passages for the test run, and 1706.03762 has page-numbered passages.

Phase 5: Evidence-first answering (1-2 days)

Files: new `evidence.py`, edit `research_workflow.py`, `nli_verifier.py`.

1. For each sub-question: FAISS top 30 → cross-encoder rerank → top 5 passages.
2. **Candidate claims come from the passages, not from the LLM.** Option A (simple, recommended first): use the best sentence of each top passage as the claim text. Option B: ask the LLM to rewrite a passage into one short factual statement, then re-verify it (Phase 5.4).
3. NLI: premise = best 1-2 sentence window around the match (not the whole chunk); hypothesis = claim. Run on GPU, batch size 16.
4. Verification of rewritten claims: the NLI must return entailment ≥ 0.80 against the source passage, or the claim is dropped.
5. Thresholds (tune on the test set, start here): entail ≥ 0.80 → verified; 0.50-0.80 → partial; contradiction ≥ 0.70 → contradicted; else unsupported.
6. Fix `nli_verifier.py`: the heuristic fallback returns **NEUTRAL** (never ENTAILS); fix the `\\b` regex; log when the model fails to load; set `device=0` if CUDA is available.
7. **Gate**: need at least 1 verified claim (factual) or at least 3 verified (review). Otherwise `insufficient_evidence` with a real reason from `stage_stats` (e.g. "0 passages indexed", "no paper matched title").

Phase 6: Answer writing (half a day)

Files: `local_llm_service.py` (rewrite, much smaller).

- Input: verified claims + their citations only. Prompt: "Rewrite these verified statements into a short answer. Add no new facts. Keep the [n] markers."
- Output is plain text with [n], **not** a large JSON. Build the comparison table, references and limitations in Python from the data, not from the LLM.
- Post-check: every sentence has a [n]; every [n] exists. Otherwise drop the sentence.
- Remove `_normalize_report_dict` default-filling entirely.
- **Done when**: the test answer names Attention Is All You Need, Vaswani et al., 2017, and self-attention, each with a clickable passage.

Phase 7: Report, PDF, UI polish (half a day)

- Status text: show "Insufficient evidence" (not `Insufficient_evidence`).
- Insufficient-evidence box: list the real reason and counts (papers retrieved, passages indexed, claims checked). Remove the template sentence "findings below are drawn from...".
- Retrieved Sources always lists the papers, even when insufficient.
- Render markdown to HTML before PDF export (no raw `###`). Add a References section matching every [n].
- Use the Deep/Standard/Quick counts from one shared constant.

Phase 8: Tests (1 day, write alongside each phase)

Test	Type	Assertion
query planner	unit (mock LLM)	no truncation, no filler terms, includes "Attention Is All You Need"

retry	unit (respx)	429 then 200 → success; 429 ×4 → error recorded, not []
ranker	unit	withdrawn dropped; exact-title paper ranked first
chunker	unit	page numbers kept; refs section removed
NLI	unit	known entail / contradict / neutral pairs; fallback never returns ENTAILS
answer post-check	unit	sentence without [n] is dropped
e2e Transformer	integration (live, marked slow)	1706.03762 retrieved; ≥1 verified claim; answer contains "self-attention"; counts in badges = counts in report
e2e set	integration	ResNet, LoRA, RAG, YOLO each retrieve their original paper
"no evidence"	integration	nonsense question → <code>insufficient_evidence</code> with a real reason

6. Config (backend/ .env)

```

LLM_PROVIDER=ollama
OLLAMA_URL=http://localhost:11434
OLLAMA_MODEL=qwen2.5:3b-instruct
SEMANTIC_SCHOLAR_API_KEY=<your key>
EMBEDDING_MODEL=BAAI/bge-small-en-v1.5
RERANK_MODEL=cross-encoder/ms-marco-MiniLM-L-6-v2
NLI_MODEL=MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli
NLI_DEVICE=cuda
NLI_ENTAIL_THRESHOLD=0.80
DEPTH_COUNTS={"Quick":6,"Standard":12,"Deep":24}

```

GPU note: run the LLM (Ollama), NLI and reranker one after another, not at once, if you have about 4 GB of VRAM. Unload the LLM between stages or run NLI on CPU if you hit out-of-memory errors.

7. Build order and time

Order	Phase	Est.
1	0 Clean repo	0.5 h
2	1 UI single source of truth	3 h
3	2 Observability	2 h
4	3 Retrieval	1 day

5	4 Extraction + index	1 day
6	5 Evidence + NLI	1.5 days
7	6 Answer writing	0.5 day
8	7 Report/PDF polish	0.5 day
9	8 Tests (in parallel)	1 day

Total: roughly 6-7 working days alone. After each phase, run the test question and record the debug output.

8. Definition of done

- Test question returns the correct paper, year, authors and key idea, each with a page-numbered passage.
- Badge numbers = report numbers = database row.
- Every [n] has a reference; PDF has no raw markdown.
- Nonsense question → honest "insufficient evidence" with a stated reason.
- Killing the Semantic Scholar key produces a visible warning in the report, not a silent drop.
- No API key in the frontend bundle. No fake content anywhere in the code.

9. Risks

Risk	Mitigation
Small local LLM gives bad JSON	Force JSON mode; keep LLM tasks tiny; fallback keyword planner
arXiv/S2 rate limits	Retry, cache, S2 key, 3 s delay
PDF extraction fails	Abstract fallback passage, flagged in the report
NLI too strict or lenient	Tune thresholds on the e2e set; log all scores
GPU memory (4 GB)	Sequential model loading, smaller models, NLI on CPU
Scope creep	Finish Phases 1-6 for the Transformer question before adding features

